

Deep Learning — Sesi 4

Framework Debugging Exercise: PyTorch Training dan Evaluation Mode

Kode Mata Kuliah: SDA2110

Sub-CPMK: Menggunakan framework Deep Learning.

Bentuk: Tugas Praktik

Bobot: 5%

Tujuan Pembelajaran

Setelah menyelesaikan latihan ini, mahasiswa mampu:

1. Menjelaskan perbedaan antara `model.train()` dan `model.eval()` .
2. Mengidentifikasi kesalahan penggunaan mode model dalam training loop.
3. Memeriksa shape, dtype, dan device pada input, label, model, dan output.
4. Menjelaskan fungsi `zero_grad()` , `backward()` , dan `optimizer.step()` .
5. Memperbaiki training step dan memberikan alasan teknis terhadap perubahan tersebut.

Instruksi

Gunakan file Python berikut:

DL_Sesi04_Framework_Debugging_Exercise.py

Jalankan program dan amati output yang dihasilkan oleh fungsi berikut:

```
inspect_batch(...)
broken_training_step(...)
corrected_training_step(...)
```

Kemudian, lakukan analisis terhadap fungsi `broken_training_step()` dan `corrected_training_step()` .

Mahasiswa diperbolehkan menjalankan program beberapa kali untuk memahami perubahan output. Namun, setiap jawaban harus disertai alasan teknis berdasarkan kode dan konsep PyTorch.

Pertanyaan

A. Analisis Kode

1. Analisis `broken_training_step()`

Perhatikan kode berikut:

```
def broken_training_step(model, optimizer, criterion, X, y):  
    model.eval()  
    optimizer.zero_grad()  
    prediction = model(X)  
    loss = criterion(prediction, y)  
    loss.backward()  
    optimizer.step()  
    return loss.item()
```

Apa masalah utama dari penggunaan:

```
model.eval()
```

di dalam fungsi tersebut?

Jelaskan:

- Fungsi `model.eval()` .
- Mode yang seharusnya digunakan saat training.
- Jenis layer yang perilakunya dapat berubah antara training dan evaluation.
- Mengapa masalah ini dapat memengaruhi hasil training.

2. Perbaiki Kode

Ubahlah fungsi `broken_training_step()` sehingga menggunakan mode yang sesuai untuk proses training.

Tuliskan versi kode yang telah diperbaiki dan jelaskan alasan perubahan tersebut.

Petunjuk:

```
model.train()
```

3. Apakah `model.eval()` Menghentikan Gradient?

Benar atau salah:

Pemanggilan `model.eval()` secara otomatis menghentikan perhitungan gradient dan membuat parameter model tidak dapat diperbarui.

Jelaskan jawaban Anda dengan membedakan fungsi berikut:

```
model.eval()  
torch.no_grad()
```

4. Urutan Training Step

Jelaskan fungsi dan urutan operasi berikut:

```
optimizer.zero_grad()  
prediction = model(X)  
loss = criterion(prediction, y)  
loss.backward()  
optimizer.step()
```

Gunakan tabel berikut dalam jawaban Anda:

Operasi	Fungsi
<code>optimizer.zero_grad()</code>	
<code>model(X)</code>	
<code>criterion(prediction, y)</code>	
<code>loss.backward()</code>	
<code>optimizer.step()</code>	

5. Risiko Melupakan `zero_grad()`

Apa yang dapat terjadi jika baris berikut dihilangkan?

```
optimizer.zero_grad()
```

Jelaskan mengapa gradient dapat terakumulasi dari beberapa mini-batch dan bagaimana hal tersebut dapat memengaruhi proses pembelajaran.

B. Pemeriksaan Input dan Output

Perhatikan fungsi berikut:

```
def inspect_batch(model, X, y):
    print("input shape:", tuple(X.shape), "dtype:", X.dtype, "device:", X.device)
    print("label shape:", tuple(y.shape), "dtype:", y.dtype, "device:", y.device)
    print("model device:", next(model.parameters()).device)

    with torch.no_grad():
        prediction = model(X)

    print("prediction shape:", tuple(prediction.shape), "dtype:", prediction.dtype)
```

6. Analisis Shape

Berdasarkan kode program, tentukan:

1. Shape `x` .
2. Shape `y` .
3. Shape output model.
4. Jumlah fitur input.
5. Jumlah output model.

Jelaskan mengapa shape label dibuat seperti berikut:

```
y = torch.randint(0, 2, (16, 1)).float()
```

7. Analisis Dtype

Mengapa label diubah menjadi tipe floating-point?

```
y = torch.randint(0, 2, (16, 1)).float()
```

Jelaskan hubungan antara dtype label dan loss function berikut:

```
criterion = nn.BCEWithLogitsLoss()
```

8. Analisis Device

Mengapa model dan tensor input harus berada pada device yang sama?

Berikan contoh masalah yang dapat muncul apabila:

```
model berada di GPU
X berada di CPU
```

Bagaimana cara memindahkan tensor atau model ke device yang sesuai?

9. Analisis Output Layer

Model yang digunakan adalah:

```
model = nn.Sequential(
    nn.Linear(2, 8),
    nn.ReLU(),
    nn.Linear(8, 1)
)
```

Jawab pertanyaan berikut:

- 1. Mengapa layer pertama memiliki input size 2 ?
- 2. Mengapa hidden layer memiliki 8 unit?
- 3. Mengapa output layer memiliki 1 unit?
- 4. Apakah model tersebut menggunakan sigmoid secara eksplisit?
- 5. Mengapa BCEWithLogitsLoss() tetap dapat digunakan tanpa menambahkan nn.Sigmoid() pada model?

C. Perbandingan Training dan Evaluation

10. Perbandingan train() dan eval()

Lengkapi tabel berikut:

Aspek	model.train()	model.eval()
Tujuan penggunaan		
Digunakan saat		
Dropout		
BatchNorm		
Gradient tracking		

Perhatikan bahwa model.eval() tidak sama dengan torch.no_grad() . Keduanya mengontrol hal yang berbeda.

11. Mengapa Evaluation Menggunakan `torch.no_grad()` ?

Jelaskan alasan penggunaan berikut saat melakukan validation atau test:

```
with torch.no_grad():  
    prediction = model(X)
```

Sebutkan minimal dua manfaatnya.

12. Membuat Fungsi Evaluation

Buatlah fungsi berikut:

```
def evaluate_step(model, criterion, X, y):  
    pass
```

Fungsi tersebut harus:

1. Mengaktifkan evaluation mode.
2. Menonaktifkan gradient tracking.
3. Menghasilkan prediksi.
4. Menghitung loss.
5. Menghitung binary accuracy.
6. Mengembalikan nilai loss dan accuracy.

Gunakan pola berikut sebagai panduan:

```
model.eval()  
  
with torch.no_grad():  
    ...
```

D. Eksperimen dan Interpretasi

13. Membandingkan Fungsi Broken dan Corrected

Jalankan kedua fungsi berikut:

```
broken_training_step(...)  
corrected_training_step(...)
```

Catat:

- Loss dari fungsi broken.
- Loss dari fungsi corrected.
- Mode model sebelum dan sesudah setiap fungsi.
- Apakah parameter model berubah?
- Apakah hasil tersebut cukup untuk menyimpulkan bahwa fungsi corrected selalu lebih baik?

Jelaskan keterbatasan perbandingan tersebut.

Catatan penting: membandingkan dua loss secara langsung tidak selalu fair apabila fungsi dijalankan berurutan pada model dan optimizer yang sama.

14. Mengapa Model Kecil Dapat Tetap Memiliki Masalah?

Model yang digunakan sangat kecil:

```
nn.Linear(2, 8)
```

Apakah model kecil pasti bebas dari bug?

Jelaskan minimal tiga masalah yang tetap dapat terjadi, misalnya:

- Shape input tidak sesuai.
- Label memiliki dtype yang salah.
- Model dan data berada pada device yang berbeda.
- Loss function tidak cocok dengan output model.
- Mode training dan evaluation tertukar.
- Gradient tidak dihapus.
- Learning rate tidak sesuai.

15. Membuat Checklist Debugging

Buat checklist minimal delapan pemeriksaan yang harus dilakukan sebelum menjalankan training panjang.

Contoh:

```
[ ] Shape input sudah diperiksa  
[ ] Shape label sudah diperiksa  
[ ] Dtype input dan label sudah sesuai  
[ ] Model dan data berada pada device yang sama
```

Tambahkan minimal empat pemeriksaan lain.

E. Pertanyaan Reflektif

16. Penjelasan Teknis

Jelaskan dengan kalimat Anda sendiri:

Mengapa kode yang tidak menghasilkan error belum tentu merupakan kode yang benar?

Berikan satu contoh dari latihan ini.

17. Peran Framework

PyTorch menyediakan automatic differentiation dan training utilities.

Mengapa mahasiswa tetap perlu memahami hal-hal berikut?

- Forward pass.
- Loss function.
- Backward pass.
- Optimizer update.
- Training mode.
- Evaluation mode.
- Shape dan dtype.

18. Kesimpulan

Tuliskan kesimpulan sepanjang **150–250 kata** yang menjelaskan:

1. Masalah utama pada `broken_training_step()` .
2. Perbaikan yang dilakukan pada `corrected_training_step()` .
3. Perbedaan `model.train()` , `model.eval()` , dan `torch.no_grad()` .
4. Pentingnya pemeriksaan shape, dtype, device, dan gradient.
5. Satu pelajaran penting yang Anda peroleh dari latihan ini.

Artefak yang Dikumpulkan

Mahasiswa harus mengumpulkan:

1. File Python yang telah diperbaiki.
2. Implementasi fungsi `evaluate_step()` .
3. Jawaban pertanyaan nomor 1–18.
4. Screenshot atau salinan output program.
5. Checklist debugging.
6. Kesimpulan reflektif sepanjang 150–250 kata.

Struktur pengumpulan yang disarankan:

```
nama_nim/  
├── DL_Sesi04_Framework_Debugging_Exercise.py  
├── jawaban.md  
├── output.txt  
├── screenshot/  
│   └── hasil_program.png
```

Rubrik Penilaian

Total: **20 poin**

Komponen	Poin
Identifikasi dan perbaikan masalah <code>model.eval()</code>	4
Penjelasan training loop dan gradient	4
Analisis shape, dtype, dan device	4
Implementasi <code>evaluate_step()</code>	4
Eksperimen, evidence, dan interpretasi	2
Refleksi dan komunikasi teknis	2
Total	20

Kriteria Jawaban Baik

Jawaban yang baik harus:

- Menjelaskan bahwa `model.eval()` mengubah perilaku layer tertentu, tetapi tidak secara otomatis menghentikan gradient.
- Membedakan `model.eval()` dari `torch.no_grad()` .
- Menjelaskan urutan `zero_grad()` → `forward` → `loss` → `backward` → `step` .
- Menyertakan alasan teknis, bukan hanya menyalin kode.
- Menampilkan output atau evidence dari program.
- Menunjukkan pemahaman terhadap `shape`, `dtype`, `device`, dan `loss function`.
- Mengakui bahwa angka `loss` dapat berbeda karena model, state optimizer, dan urutan eksekusi.

Referensi

- PyTorch Documentation — `Module.train()` dan `Module.eval()` .
- PyTorch Documentation — `torch.no_grad()` .
- PyTorch Documentation — `BCEWithLogitsLoss` .
- Chollet, F. (2021). *Deep Learning with Python*, 2nd ed. Manning, Bab 3.
- Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. O'Reilly, Bab 10.
- Dokumentasi resmi PyTorch mengenai `tensors`, `nn.Module` , `autograd`, `optimizer`, dan `training loop`.

Catatan: Penggunaan AI assistant harus mengikuti kebijakan dosen dan aturan integritas akademik. Mahasiswa tetap bertanggung jawab memahami, menguji, dan menjelaskan kode yang dikumpulkan.